

PicoRV32 + UART + CNN TRÊN DE10-LITE

Học phần:

Thực Tập

Giảng viên hướng dẫn: TS. Lê Minh Tuấn

ThS.Vương Viết Thao

Sinh viên thực hiện: Phan Quang Hưng - B22DCDT149

Vũ Tiến Quốc - B22DCDT253

Mục tiêu và phần việc đã thực hiện

Kết quả hiện tại có ba lớp: UART lỗi, PicoRV32 chạy firmware và giao thức truyền ảnh 784 byte.

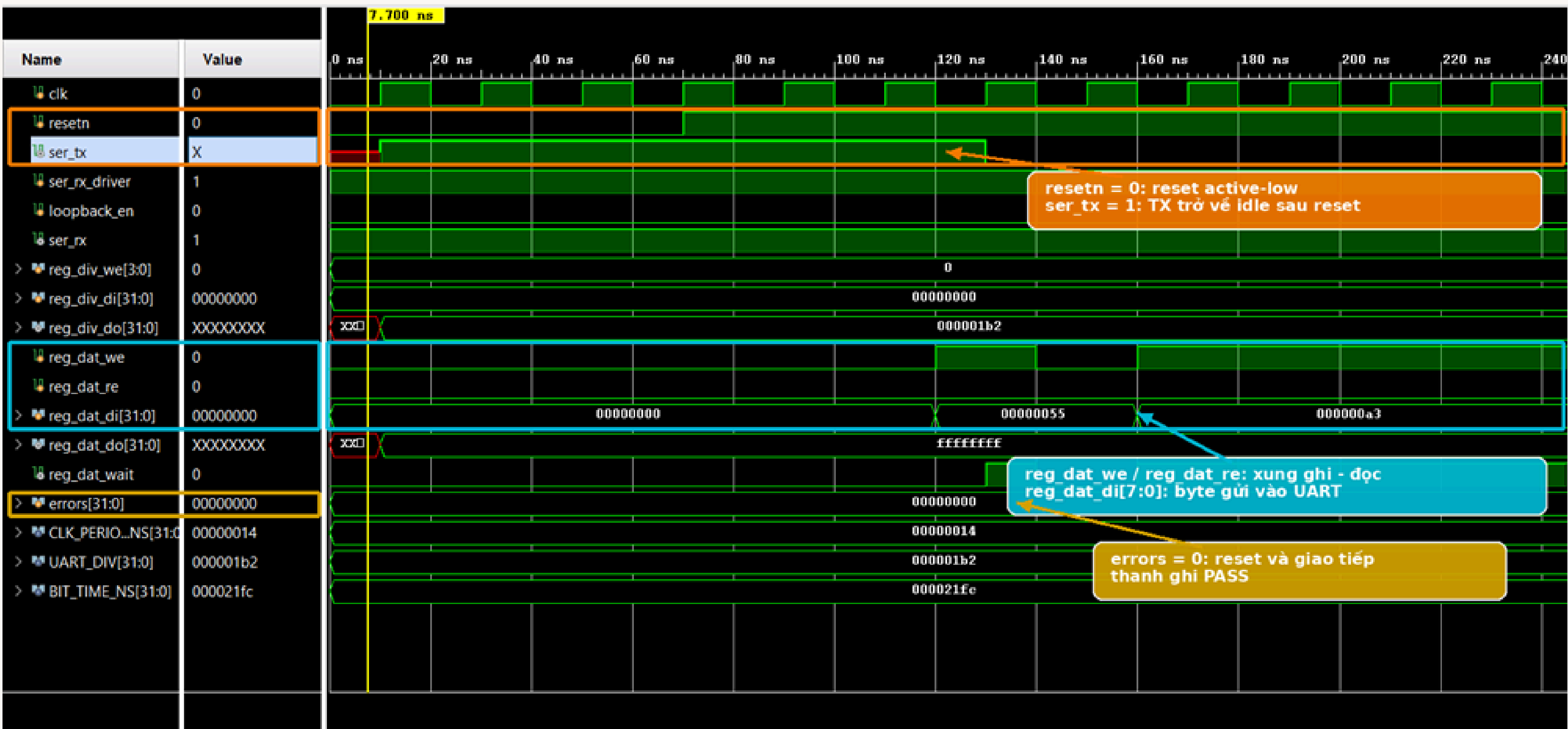
- Kiểm chứng reset, TX, RX, loopback và cờ bận của simpleuart.
- Kiểm chứng PicoRV32 boot firmware từ BRAM, truy cập GPIO và phát banner UART.
- Kiểm chứng end-to-end: nhận 784 byte ảnh, echo từng byte, tính SUM/XOR và phát hiện dữ liệu bị sửa.
- Thiết kế tích hợp: PicoRV32 truy cập CNN accelerator qua MMIO và AXI4-Stream.

UART với Risc V lõi picoRV32

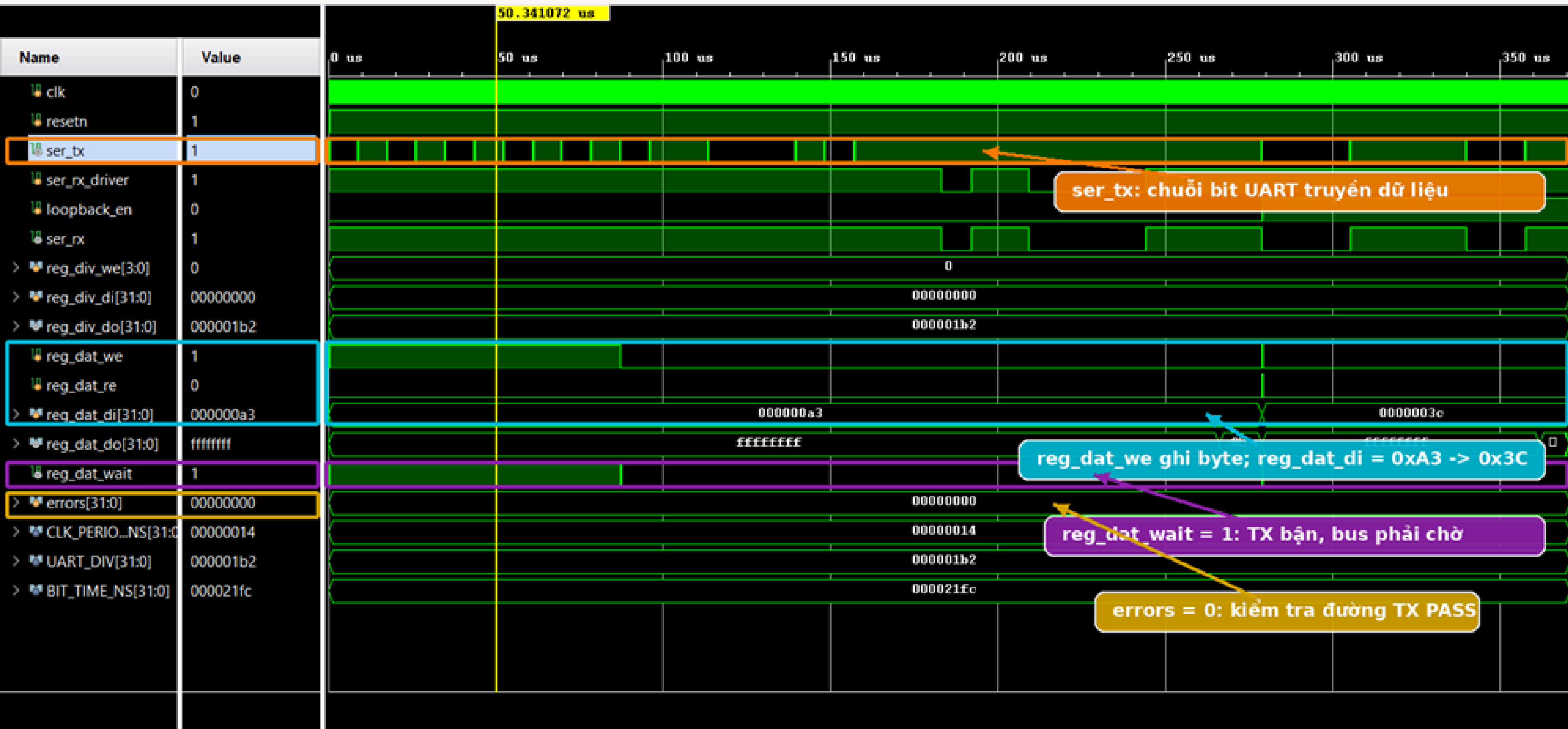
Cấu trúc RTL:

- top.v: kết nối chân clock, KEY, UART, SW và LEDR.
- picosoc.v: tổ chức PicoRV32, BRAM, bus MMIO và simpleuart.
- simpleuart.v: UART TX/RX, divider, thanh ghi dữ liệu và cờ wait.
- firmware8.hex: firmware RISC-V nạp vào BRAM.
- tb_simpleuart.v: unit test reset, TX, busy, RX và loopback.
- tb_top_uart.v / tb_pico_uart_image.v: integration test CPU, firmware và giao thức UART

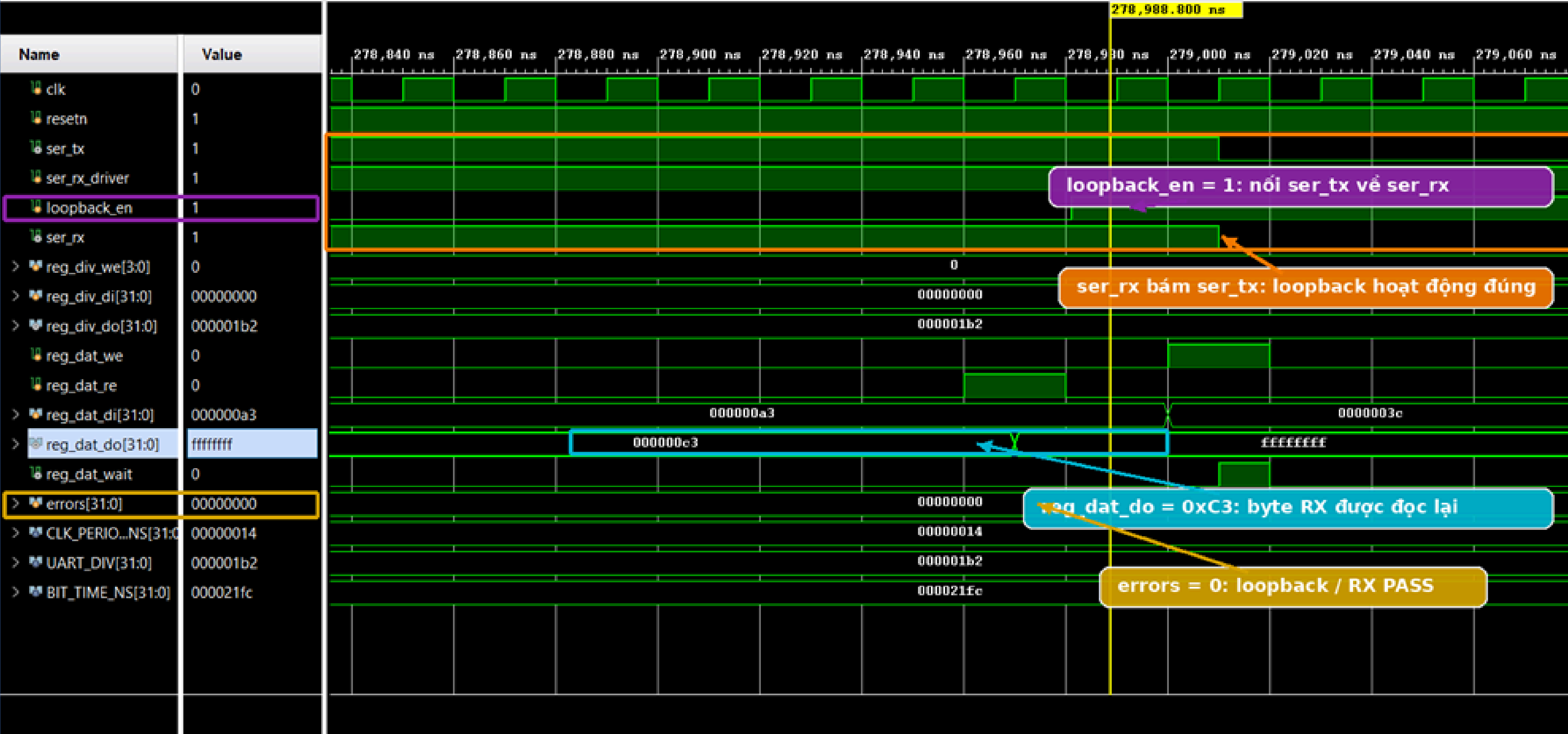
Reset active-low khởi tạo UART về trạng thái xác định



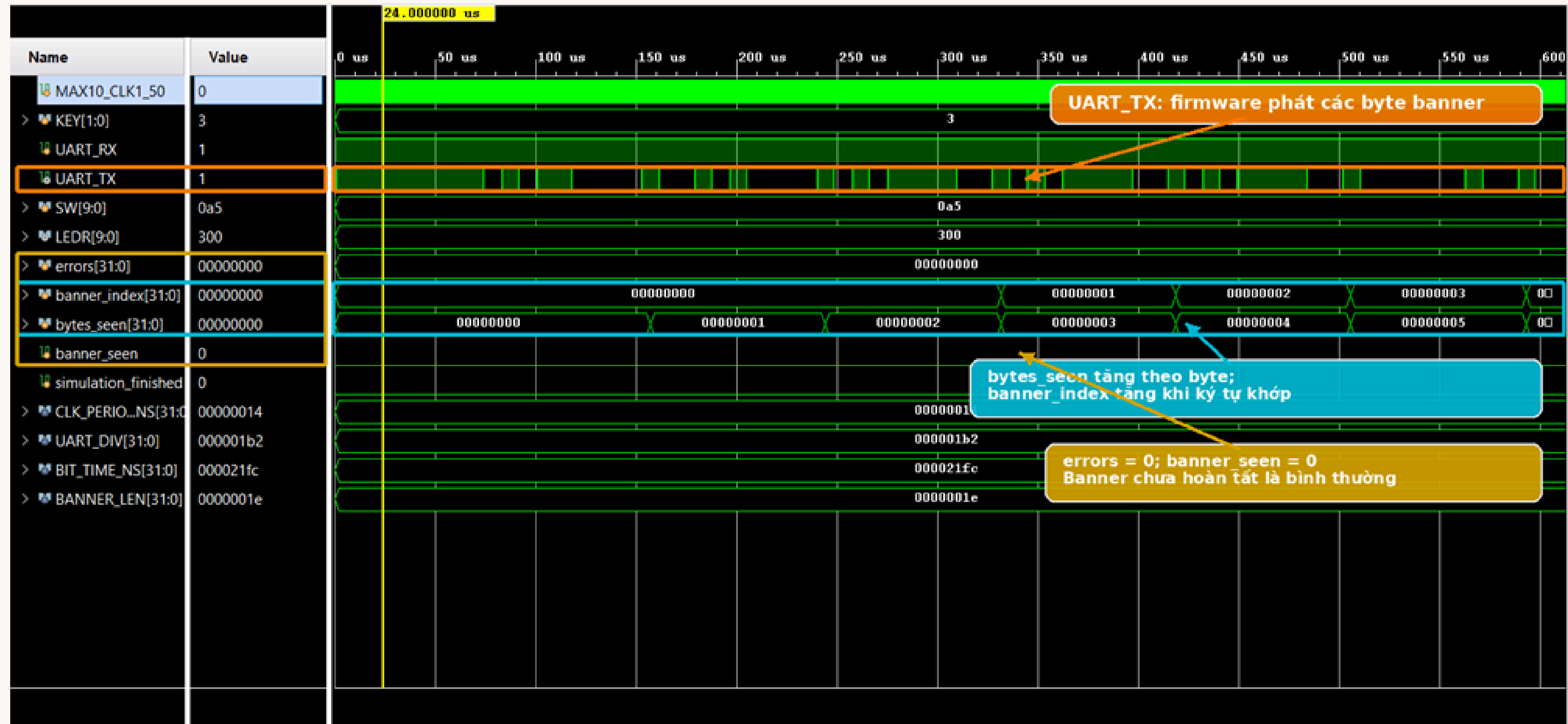
Ghi dữ liệu kích hoạt UART transmit và chờ wait bảo vệ bus



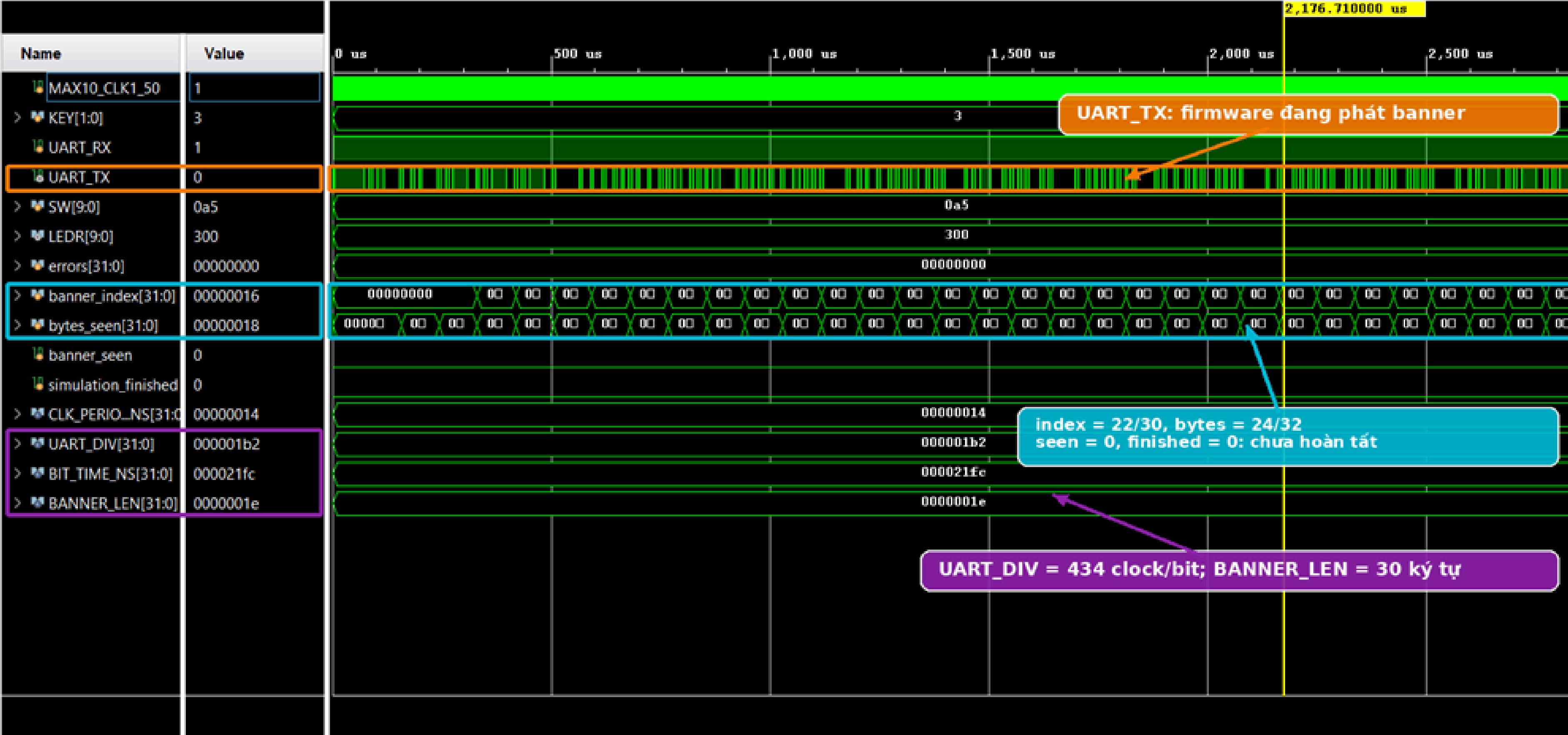
Loopback xác nhận UART receive và thanh ghi dữ liệu đọc



PicoRV32 boot firmware và bắt đầu phát banner UART



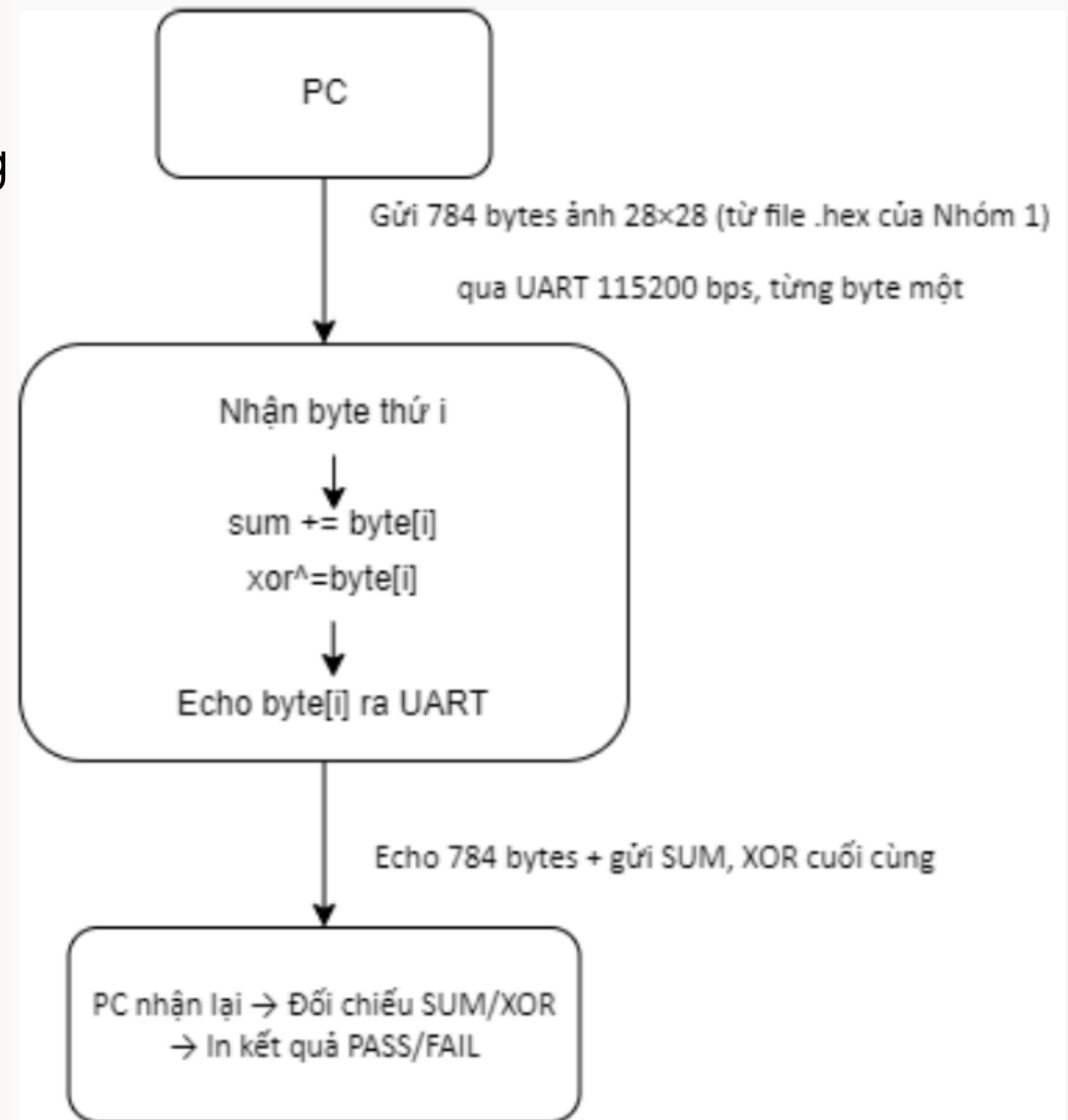
Theo dõi tiến trình khớp banner UART



KIỂM THỬ GIAO TIẾP UART ĐỘC LẬP

Mục tiêu

- Ảnh đầu vào gồm 784 byte, tương đương 28×28 pixel grayscale.
- Testbench chờ firmware gửi READY 784 trước khi truyền dữ liệu.
- PicoRV32 nhận ảnh qua UART_RX, echo từng byte qua UART_TX.
- Hai phía tính SUM và XOR để kiểm tra toàn vẹn dữ liệu.
- Có ca chuẩn và ca cố ý đảo một bit tại pixel số 100.



FSM của testbench điều khiển một phiên truyền ảnh

- PHASE_RESET: giữ reset cho DUT.
- PHASE_WAIT_READY: chờ firmware phát READY 784.
- PHASE_RECEIVE: gửi 784 pixel qua UART_RX.
- PHASE_ECHO: nhận và đối chiếu 784 byte echo.
- PHASE_CHECK: kiểm SUM/XOR và CHECKSUM PASS/FAIL.
- PHASE_DONE_PASS hoặc PHASE_DONE_FAIL: kết luận testcase.

PHASE[3:0]	Hex	Giai đoạn
0000	0	PHASE_RESET – đang reset
0001	1	PHASE_WAIT_READY – chờ firmware gửi "READY 784"
0010	2	PHASE_READY – đã nhận được READY
0011	3	PHASE_RECEIVE – PC đang gửi 784 pixel
0100	4	PHASE_ECHO – đang nhận và kiểm tra 784 byte echo
0101	5	PHASE_CHECK – kiểm tra SUM/XOR và checksum
0110	6	PHASE_DONE_PASS – hoàn thành và PASS
0111	7	PHASE_DONE_FAIL – hoàn thành nhưng FAIL

Waveform truyền 784 pixel: từ chờ READY đến RECEIVE và ECHO



KIỂM THỬ GIAO TIẾP UART ĐỘC LẬP

Kết quả

```
PS D:\Python\internship\01> & C:\Users\ASUS\AppData\Local\Programs\Python\
● Số pixel: 784
SUM      : 0x9284
XOR      : 0x64
Cổng UART: COM3 @ 115200
Nhấn giữ KEY[0] khoảng 0.2 giây rồi THẢ ra để FPGA boot...
FPGA: === RISC-V IMAGE UART TEST ===
FPGA: READY 784
FPGA: ECHO 784
BYTE-BY-BYTE: PASS
FPGA: SUM=0x00009284 XOR=0x64
FPGA: CHECKSUM PASS
KẾT QUẢ CUỐI: PASS
○ PS D:\Python\internship\01> █
```

784

byte ảnh nhận và echo

0x9284

SUM ảnh gốc

0x64

XOR ảnh gốc

PASS

so sánh đầu-cuối

KIỂM THỬ GIAO TIẾP UART ĐỘC LẬP

3. Thử nghiệm

ẢNH GỐC

```
PS D:\Python\internship\01> & C:\Users\ASUS\AppData\Local\Microsoft\Windows\Fonts\
• Số pixel: 784
SUM      : 0x9284
XOR      : 0x64
Cổng UART: COM3 @ 115200
Nhấn giữ KEY[0] khoảng 0.2 giây rồi THẢ ra để FPGA
FPGA: === RISC-V IMAGE UART TEST ===
FPGA: READY 784
FPGA: ECHO 784
BYTE-BY-BYTE: PASS
FPGA: SUM=0x00009284 XOR=0x64
FPGA: CHECKSUM PASS
KẾT QUẢ CUỐI: PASS
```

SUM=0x9284 • XOR=0x64
CHECKSUM PASS

SỬA 1 PIXEL

```
PS D:\Python\internship\01> & C:\Users\ASUS\AppData\Local\Microsoft\Windows\Fonts\
Số pixel: 784
SUM      : 0x9285
XOR      : 0x65
Cổng UART: COM3 @ 115200
Nhấn giữ KEY[0] khoảng 0.2 giây rồi THẢ ra để FPGA boot..
FPGA: === RISC-V IMAGE UART TEST ===
FPGA: READY 784
FPGA: ECHO 784
BYTE-BY-BYTE: PASS
FPGA: SUM=0x00009285 XOR=0x65
FPGA: CHECKSUM FAIL

LỖI: Bài test UART ảnh không đạt
Kiểm tra: nan đúng output files\nicosoc sof mỗi phút: rev
```

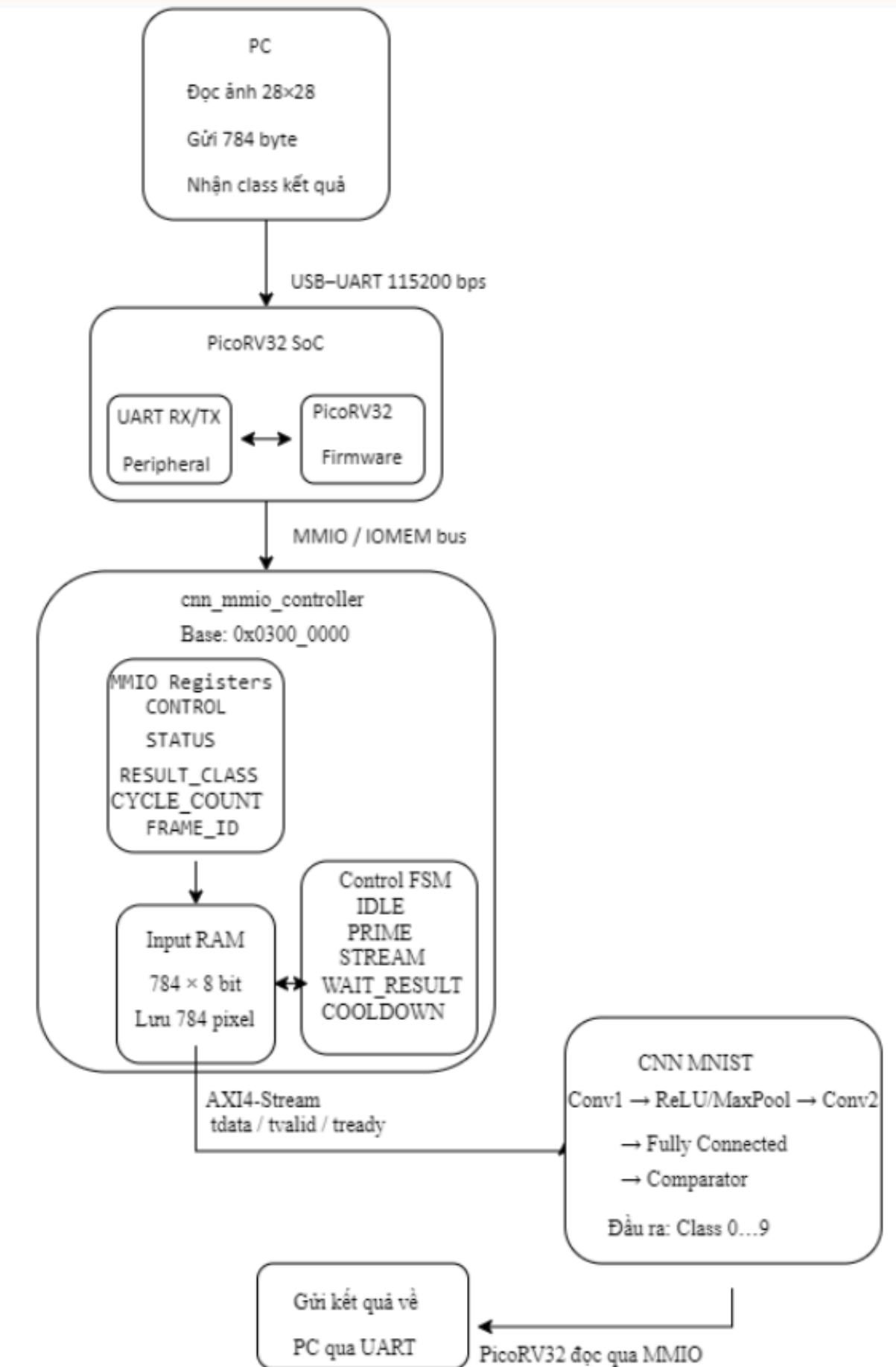
SUM=0x9285 • XOR=0x65
CHECKSUM FAIL

Kết luận: dữ liệu được kiểm tra thật, không phải PASS cố định

Kiến trúc tích hợp PicoRV32 với CNN accelerator

Sơ đồ khối tổng thể

- PC/Testbench và PicoRV32 giao tiếp qua UART.
- PicoRV32 và cnn_mmio_controller giao tiếp qua IOMEM/MMIO.
- cnn_mmio_controller quản lý thanh ghi điều khiển, trạng thái và Input RAM.
- Controller đưa 784 pixel tới CNN qua AXI4-Stream.
- CNN trả class dự đoán và trạng thái hoàn tất về controller để CPU đọc.

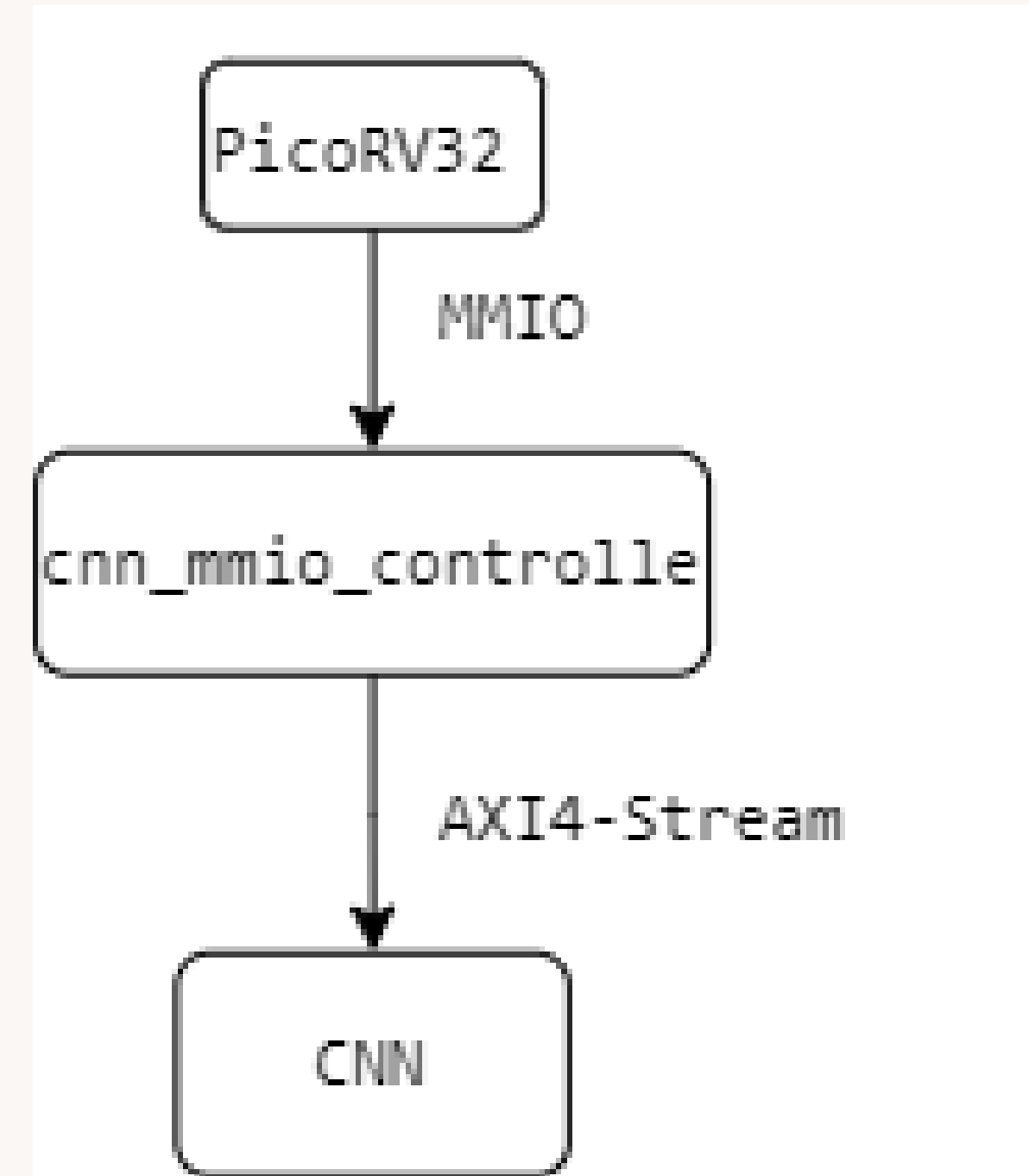


TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Khối trung gian Controller

-Cung cấp thanh ghi MMIO

- CPU dùng CONTROL để START, CLEAR_ALL và ACK_DONE.
- CPU đọc STATUS để biết READY, BUSY, DONE, ERROR và INPUT_FULL.
- CPU ghi từng pixel vào INPUT_DATA8, controller tăng INPUT_COUNT.
- Controller giữ RESULT_CLASS, CYCLE_COUNT và FRAME_ID để CPU đọc ổn định.
- Controller điều khiển AXI4-Stream tới CNN thay vì để CPU can thiệp từng chu kỳ.



TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Khối trung gian Controller

Ảnh phải được nhận đủ vào RAM trước khi CNN xử lý để tránh phụ thuộc trực tiếp vào tốc độ UART.

- Dung lượng: $784 \times 8 \text{ bit} = 6272 \text{ bit} = 784 \text{ byte}$.
- word 0 chứa pixel[0], word 783 chứa pixel[783].
- UART nhận byte theo tốc độ 115200 baud.
- CNN có thể lấy pixel theo tốc độ clock phần cứng sau khi Input RAM đã đầy.
- Tách RAM chương trình PicoRV32 khỏi RAM đầu vào CNN để dữ liệu không xung đột.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Khối trung gian Controller

Mỗi giao dịch MMIO ghi pixel vừa lưu dữ liệu vừa cập nhật tiến trình frame.

- CPU ghi: INPUT_DATA8 = pixel 8 bit.
- Controller ghi pixel vào cnn_input_ram tại địa chỉ INPUT_COUNT.
- INPUT_COUNT tăng: $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 784$.
- Khi nhận đủ pixel: INPUT_FULL = 1.
- READY chỉ được bật khi frame đầu vào hợp lệ và controller có thể nhận START.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Khối trung gian Controller

Controller kiểm tra điều kiện đầu vào trước khi cho phép CNN chạy.

Khi CPU ghi START, controller kiểm tra:

- Đã đủ 784 pixel chưa? `INPUT_FULL = 1`.
- CNN có đang bận không? `BUSY = 0`.
- CNN có sẵn sàng nhận dữ liệu không? ví dụ `s_axis_tready`.

Nếu hợp lệ: `BUSY = 1`, `DONE = 0` và FSM bắt đầu chạy.

Nếu không hợp lệ: giữ trạng thái hoặc thiết lập `ERROR_CODE` tùy thiết kế.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Khối trung gian Controller

Kết quả CNN được giữ ổn định trong thanh ghi; CPU không phải bắt đúng một xung ngăn ở đầu ra CNN.

- Khi CNN phát class cuối, controller chốt RESULT_CLASS.
- CYCLE_COUNT lưu số chu kỳ suy luận.
- DONE = 1 và BUSY = 0.
- FRAME_ID tăng 1 để đánh dấu frame đã xử lý.
- CPU đọc kết quả qua MMIO; sau đó có thể ACK_DONE hoặc bắt đầu frame mới.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

FSM điều khiển stream 784 pixel vào CNN

FSM xử lý đặc điểm của RAM đọc đồng bộ và pipeline CNN để dữ liệu không lệch một chu kỳ.

- IDLE: chờ đủ ảnh và lệnh START.
- PRIME: chờ một clock để pixel đầu tiên từ Input RAM ổn định.
- STREAM: lần lượt gửi pixel[0] đến pixel[783].
- WAIT_RESULT: đã gửi đủ dữ liệu nhưng vẫn giữ BUSY để chờ class cuối.
- COOLDOWN: chờ pipeline CNN trở lại sẵn sàng trước khi về IDLE.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

AXI4-Stream

- Controller lần lượt đưa 784 pixel sang CNN:
pixel[0], pixel[1], ..., pixel[783]
- Mỗi pixel chỉ được chuyển khi:
tvalid = 1
tready = 1

WAIT_RESULT:

Sau khi gửi pixel thứ 784:

- Controller ngừng gửi dữ liệu.
- Tiếp tục giữ BUSY=1.
- Chờ CNN xuất class kết quả.

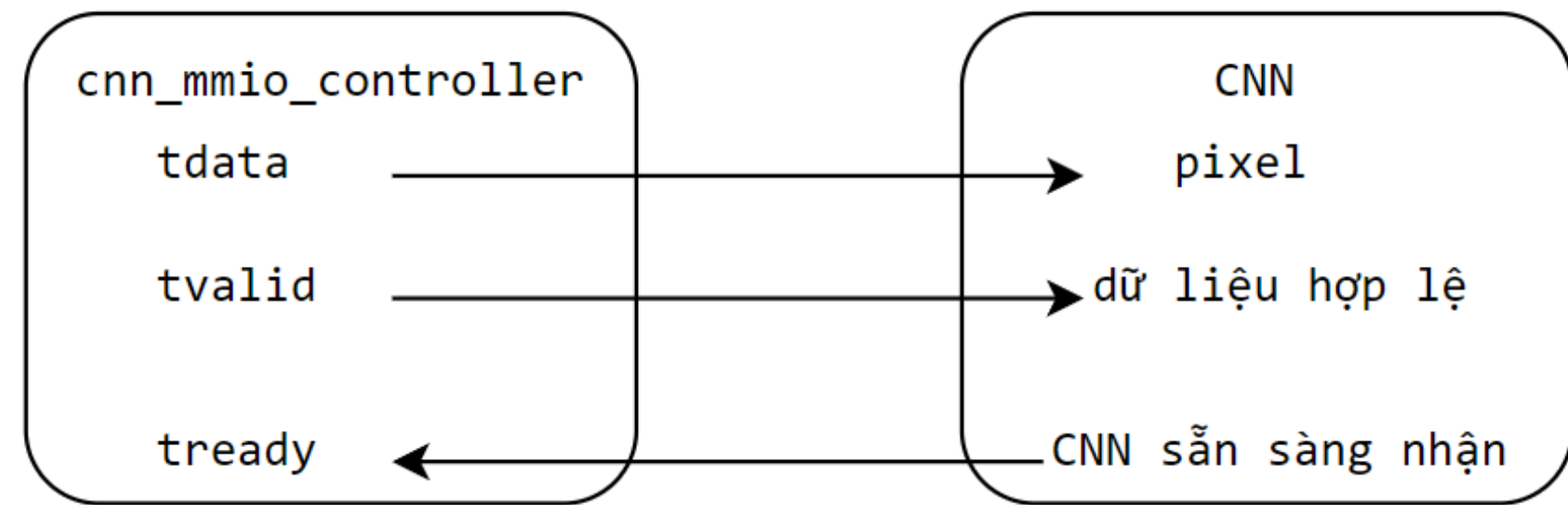
COOLDOWN:

- CNN mẫu cần thời gian xóa pipeline và trở lại trạng thái sẵn sàng.
- Controller chỉ trở về IDLE khi CNN đưa s_axis_tready lên lại.

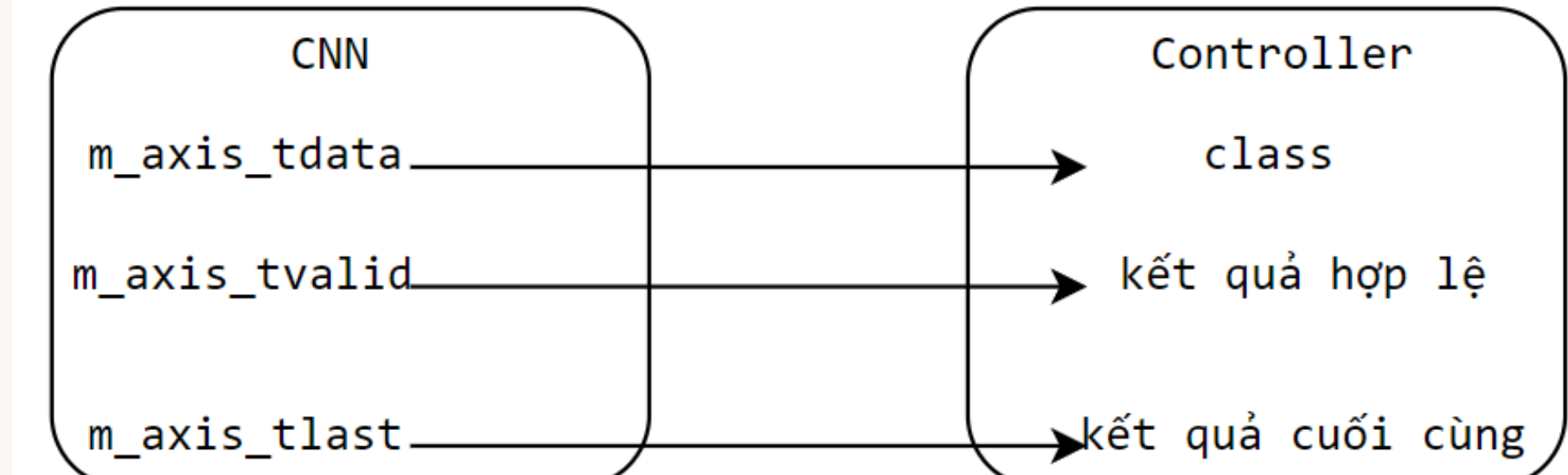
TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

5. AXI4-Stream giữa controller và CNN

controller tới CNN sử dụng AXI4-Stream:



Đầu ra CNN sử dụng một luồng khác:



Một pixel được truyền thành công tại cạnh clock khi: `tvalid && tready`

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Bản đồ thanh ghi CNN MMIO

BẢN ĐỒ THANH GHI CNN-MMIO		
Địa chỉ	Thanh ghi	Chức năng
0x0300_0000	ID_VERSION	Nhận dạng controller "CNN1"
0x0300_0004	CONTROL	START , CLEAR_ALL , ACK_DONE
0x0300_0008	STATUS	READY , BUSY , DONE , ERROR , INPUT_FULL
0x0300_000C	CONFIG	Cấu hình CNN: int8, 10 class, 28×28
0x0300_0010	INPUT_LEN	Số pixel yêu cầu: 784
0x0300_0014	INPUT_COUNT	Số pixel đã nhận
0x0300_0018	INPUT_DATA8	Ghi từng pixel 8 bit
0x0300_001C	RESULT_CLASS	Class dự đoán 0–9
0x0300_0020	CYCLE_COUNT	Số chu kỳ xử lý
0x0300_0024	ERROR_CODE	Mã lỗi
0x0300_0028	FRAME_ID	Số thứ tự ảnh đã xử lý

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

STATUS phản ánh trạng thái của CNN controller

CPU chỉ cần đọc một thanh ghi STATUS để biết controller có nhận lệnh, đang bận hay đã hoàn tất hay không.

- bit 0 - READY (0x01): có thể nhận lệnh START.
- bit 1 - BUSY (0x02): controller/CNN đang xử lý.
- bit 2 - DONE (0x04): đã có kết quả.
- bit 3 - ERROR (0x08): có lỗi.
- bit 4 - INPUT_FULL (0x10): đã nhận đủ 784 pixel.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Ví dụ giải mã **STATUS = 0x15** khi một frame hoàn tất

- $0x15 = 0001\ 0101$.
- $READY = 1$: controller sẵn sàng cho chu kỳ tiếp.
- $BUSY = 0$: không còn xử lý.
- $DONE = 1$: đã có kết quả để CPU đọc.
- $ERROR = 0$: không phát hiện lỗi.
- $INPUT_FULL = 1$: frame 784 pixel đã được nhận đủ.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Các giá trị **STATUS** thường gặp trong firmware

- 0x00: chưa có dữ liệu.
- 0x11: đã đủ 784 pixel và sẵn sàng START.
- 0x12: CNN đang xử lý.
- 0x15: CNN xử lý xong, có kết quả.
- Có bit 0x08: firmware phải đọc ERROR_CODE hoặc CLEAR_ALL.
- Firmware nên dùng phép AND theo từng bit thay vì so sánh cứng mọi giá trị STATUS.

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Kết quả

Kết quả quan sát

- CLASS=3 đúng expected=3
- CYCLES=1282 ở cả ba lần
- FRAME tăng 1 → 2 → 3
- STATUS=0x00000015
- FINAL: 3/3 repeat tests passed

```
PS D:\VSC\Microsoft VS Code> & C:\Users\ASUS\AppData\Local\Programs\Python\Python38-32\python.exe test_riscv_cnn.py [-h] --port PORT [--baud BAUD] [--input INPUT] [--no-reset-prompt]
usage: test_riscv_cnn.py [-h] --port PORT [--baud BAUD] [--input INPUT] [--no-reset-prompt]
test_riscv_cnn.py: error: the following arguments are required: --port
PS D:\VSC\Microsoft VS Code> python "E:\Quartus\Project_Quartus\04_risc_cnn\test_riscv_cnn.py"
Pixels: 784
SUM   : 0x00004D13
XOR   : 0x75
Press KEY[0] for about 0.2 s, release it, then press Enter...
FPGA: === PICORV32 + CNN SAMPLE INTEGRATION ===
FPGA: CNN_ID=0x434E4E31 CONFIG=0x080A1C1C
FPGA: READY 784
FPGA: RX SUM=0x00004D13 XOR=0x75
FPGA: RESULT CLASS=3 CYCLES=1282 FRAME=1 STATUS=0x00000015
RUN 1: class=3 expected=3 cycles=1282 frame=1 status=0x00000015 => PASS
FPGA: READY 784
FPGA: RX SUM=0x00004D13 XOR=0x75
FPGA: RESULT CLASS=3 CYCLES=1282 FRAME=2 STATUS=0x00000015
RUN 2: class=3 expected=3 cycles=1282 frame=2 status=0x00000015 => PASS
FPGA: READY 784
FPGA: RX SUM=0x00004D13 XOR=0x75
FPGA: RESULT CLASS=3 CYCLES=1282 FRAME=3 STATUS=0x00000015
RUN 3: class=3 expected=3 cycles=1282 frame=3 status=0x00000015 => PASS
FINAL: 3/3 repeat tests passed
```

TÍCH HỢP RISC-V VỚI CNN ACCELERATOR

Kết quả

Kết quả quan sát

- Độ chính xác: 96,6% — CNN nhận dạng đúng 966/1.000 ảnh, sai 34 ảnh.
- Protocol Errors = 0 : toàn bộ quá trình UART → PicoRV32 → MMIO → CNN → UART hoạt động đúng; không có lỗi STATUS, FRAME_ID hoặc số chu kỳ.
- CNN Cycles = 1282 cho mọi ảnh — thời gian xử lý CNN ổn định và không phụ thuộc nội dung ảnh. Với clock 50 MHz, tương đương khoảng 25,64 μ s/ảnh.
- Elapsed = 387,53 s — tổng thời gian end-to-end cho 1.000 ảnh, gồm truyền UART, điều khiển bằng firmware, chờ phản hồi và xử lý trên PC.
- Throughput = 2,6 ảnh/giây — tốc độ toàn hệ thống hiện bị giới hạn chủ yếu bởi UART và giao thức PC-FPGA, không phải tốc độ tính toán của CNN.

Accuracy

 966/1000
96.60% (✓ 966 correct X 34 missed)

Protocol

✓ Errors: 0

Performance

CNN Cycles	min=1282	max=1282	avg=1282
Elapsed	387.53 s		
Throughput	2.6 images/sec		

Kết luận và bước tiếp theo

- Đã PASS: reset, TX/busy, loopback/RX của UART.
- Đã PASS: PicoRV32 boot firmware và phát banner qua UART.
- Đã PASS: truyền/echo 784 byte ảnh, byte-by-byte check, SUM/XOR và negative test.
- Đã thiết kế: MMIO register map, Input RAM, control FSM, AXI4-Stream và STATUS cho CNN controller.
- Đã PASS: gắn RTL cnn_mmio_controller/CNN, chạy test vector và lưu waveform/log class kết quả.
- Đã PASS: kiểm chứng trên DE10-Lite với UART vật lý và firmware thực.
- Bước tiếp theo: tích hợp trên ...